Nama: Muhammad Rafly Romeo Nasution
Kelas: 3KA25
NPM: 10123875

MK Praktikum Unggulan Universitas Gunadarma

Mata Kuliah: Praktikum Terapan Teori Graf (Tingkat 3)

Pertemuan VI

```
graph = {
    'A' : ['B','C'],
    'B' : ['D', 'E'],
    'C' : [],
    'D' : [],
    'E' : ['F'],
    'F' : []
}
# Menambahkan kode program dari materi DFS Flow
graph_val = {"A": 10, 'B': 12, 'C': 3, 'D': 4, 'E': -6, 'F': -5}
```

```
visited = set() # Set to keep track of visited nodes.
```

```
leaf = set() # Menambahkan kode program dari materi DFS Flow
```

```
def dfs(visited, graph, node):
    if node not in visited:
        visited.add(node)
        # Menambahkan kode program dari materi DFS Flow
        if len(graph[node]) == 0:
            leaf.add(node)
        for neighbour in graph[node]:
            dfs(visited, graph, neighbour)
```

```
def leaf_sum(leaf):
    jmlh = 0
    for leaf_node in leaf:
        leaf_val = graph_val[leaf_node]
        jmlh = jmlh + leaf_val

    return jmlh
```

```
dfs(visited, graph, 'A')
```

```
leaf_sum(leaf)
```

```
2
```

Logika Program DFS Leaf Sum

1. Struktur Data

Program mendefinisikan dua hal:

- `graph` — representasi graf berarah (adjacency list), menunjukkan siapa tetangga tiap node
- `graph_val` — nilai/bobot setiap node

2. Fungsi `dfs(visited, graph, node)`

Ini adalah inti program dengan logika rekursif:

1. **Cek apakah node sudah dikunjungi** — jika sudah, fungsi langsung berhenti (mencegah loop tak terbatas)
2. **Tandai node sebagai visited** dengan `visited.add(node)`
3. **Cek apakah node adalah leaf** — jika `graph[node]` kosong (tidak punya tetangga), masukkan ke set `leaf`
4. **Rekursi ke semua tetangga** — panggil `dfs()` untuk setiap node yang terhubung

3. Fungsi `leaf_sum(leaf)`

Setelah DFS selesai, fungsi ini menjumlahkan nilai semua leaf node yang ditemukan:

- Iterasi semua node dalam set `leaf`
- Ambil nilai dari `graph_val[leaf_node]`

- Akumulasi ke variabel `jmlh`

4. Hasil Eksekusi

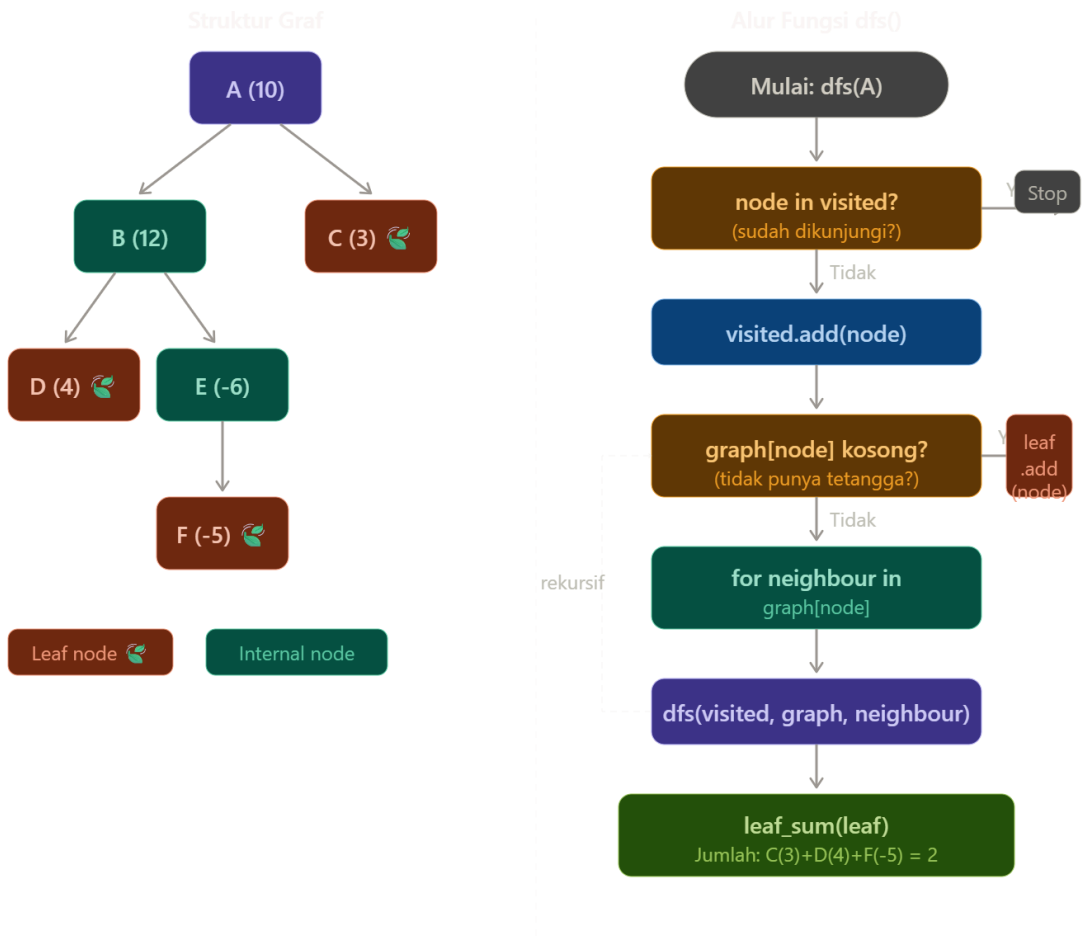
Leaf node yang ditemukan adalah **C, D, F** (tidak memiliki tetangga). Hasilnya:

```
C(3) + D(4) + F(-5) = 2
```

5. Urutan Pemanggilan DFS

```
dfs(A) → dfs(B) → dfs(D) [leaf!]
                → dfs(E) → dfs(F) [leaf!]
       → dfs(C) [leaf!]
```

> **Catatan penting:** Karena `visited` dan `leaf` adalah variabel global (`set()`), pastikan di-reset ke `set()` baru jika ingin menjalankan DFS ulang dari awal pada graf yang berbeda.



Copyright © Pengelola MK Praktikum Unggulan (Praktikum DGX), Universitas Gunadarma

https://www.praktikum-hpc.gunadarma.ac.id/